

Working with MongoDB



Shailendra Chauhan

Microsoft MVP, Technical Consultant and Corporate Trainer

What You will learn?

- Data Types
- Collections and Capped Collection
- Documents
- MongoDB CRUD Operations
- Cursor
- Querying Collections
- Schema Validation
- Indexes

Getting Started with MongoDB

Data Types

- Integer
- Boolean
- Null
- Double
- String
- Arrays
- Object
- Date
- Timestamp
- Binary data
- Object ID
- Regular expression
- Code
- Min/Max keys

Collection

- Created on-the-fly when referenced first time.
- Schema-less.
- Contains Documents.
- Supports Indexes based on one or more fields.

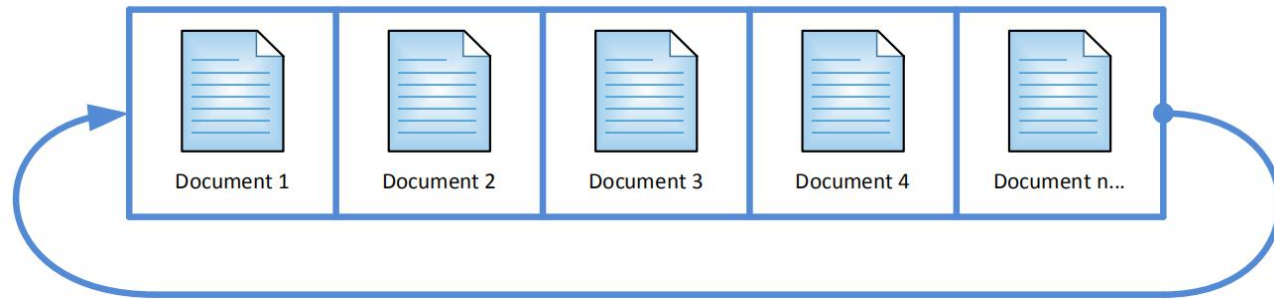
```
createCollection(name, options)
```

name is a mandatory parameter, while **options** is optional

options are capped: <boolean>, size: <number>, max: <number>

Capped Collection

- A fixed-size collection that support high-throughput operations.
- Work like a circular buffers i.e. once a collection fills its allocated space, it adds new documents by overwriting the oldest documents in the collection.



- You cannot remove a document from a capped collection.

Capped Collection Usages

- Logging (Activity logging)
- Caching (Preserving the latest items)
- Acting as a queue

Document

- Created on-the-fly when referenced first time.
- Created `_id` field which acts as Primary key in RDBMS.
- Supports Relationships using Embedded or References.
- Stores data in BSON (Binary form of JSON).

MongoDB CRUD Operations

CREATE Operation

- `db.collection.insert(<document(s)>)`
- `db.collection.insertOne(<document>)`
- `db.collection.insertMany(<documents>)`

RETRIEVE Operation

- `db.collection.find(<filter>)`
- `db.collection.findOne(<filter>)`
- `pretty()` - Shows result in proper JSON format

UPDATE Operation

- `db.collection.update(<filter>,<document>)`
- `db.collection.updateOne(<filter>,<document>)`
- `db.collection.updateMany(<filter>,<document>)`

DELETE Operation

- `db.collection.remove(<filter>)`
- `db.collection.deleteOne(<filter>)`
- `db.collection.deleteMany(<filter>)`

Find document(s) and Perform Operations

- `db.collection.findOneAndDelete(<filter>)`
- `db.collection.findOneAndUpdate(<filter>, <update>)`
- `db.collection.findOneAndReplace(<filter>, <replacement>)`
- `db.collection.findAndModify(<filter>, <document>)`

Cursor

- A cursor is a pointer to a collection of documents which is returned upon the *find()* method execution.
- By default, the cursor will be iterated automatically when the result of the query is returned. But explicitly you can go through the items returned in a cursor one by one

```
var users= db.users.find(); //return cursor to users
users.size(); //return size

users.forEach(function(u){
    print(u.name); //print name
});
```

Cursor Methods

- `count()`, `size()`
- `explain()`
- `foreach()`, `pretty()`
- `close()`, `isClosed()`
- `hasNext()`, `next()`
- `limit()`, `skip()`
- `min()`, `max()`

DEMO:

Querying Collections

Query Selectors

Query Selectors

Comparison

- \$eq, \$ne
 - \$gt, \$gte
 - \$lt, \$lte
 - \$in, \$nin

Logical

- \$and
- \$not
- \$nor
- \$or

Element

- \$exist
- \$type

Evaluation

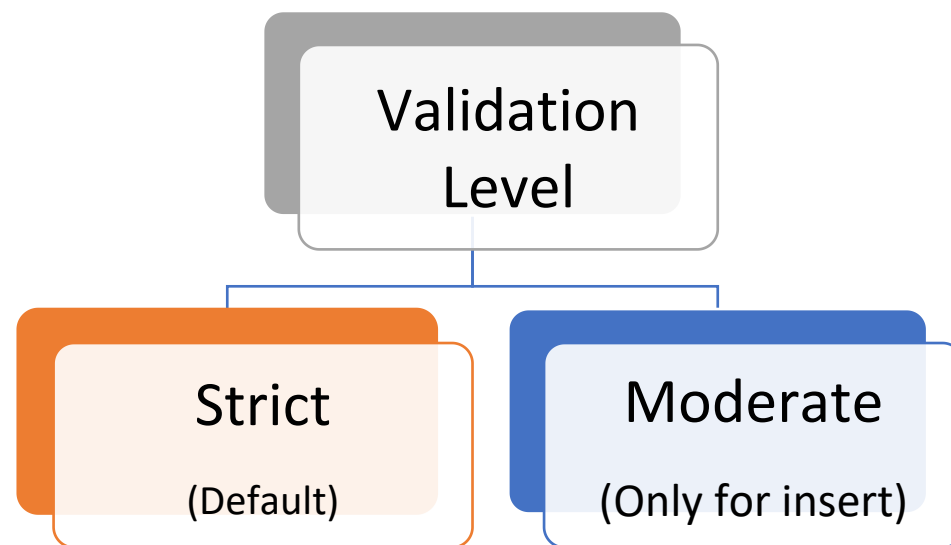
- \$regex
 - \$where
 - \$text

DEMO:

Query Selectors

Schema Validation

- Introduced with MongoDB version 3.2 .
- Helps to validate document's fields data.
- Automatically tests for each insert/update and delivers warning or error if a rule is broken.



Defining Schema Validation

Creating Collection with Schema Validation

```
db.createCollection("customers", {  
  validator: {  
    _id: { $type: "number" },  
    name: { $type: "string" },  
    age: { $type: "number", $gte: 18 }  
  }  
});
```

Creating Collection with Json Schema

```
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "age"],
      properties: {
        name: { bsonType: "string", description: "must be a string and is required" },
        age: {
          bsonType: "number", minimum: 18, maximum: 100,
          description: "must be a number between 18 & 100 and is required"
        }
      }
    }
  },
  validationLevel: "moderate"
})
```

DEMO:

Schema Validation

Schema Validation on existing Collection

```
db.runCommand({
  collMod : "employees",
  validator: { $and: [{ _id: { $gte: 1 }},
    { $or: [ { firstname: { $type: "string" }},
      { lastname: { $type: "string" } }
    ] }
  ] }
})
```

DEMO:

Schema Validation on existing Collection

Indexes

- A database index is similar to a book's index.
- Indexes organize data by a given field or fields.
- A query that doesn't use an index *scan a collection from start to end*.

DEMO:

Creating Indexes

Types of Indexes

- Single Field
- Compound Index
- Multikey Index
- Text Index
- Hash Indexes
- Geospatial Index

Learn. Build. Empower.